# Model training in practice

Maciej Kępa

# Roadmap

Data Community

## 01 AI/ML architecture - how it all fits together

Understand the full ML stack from data sources to production, including roles, cloud reference architectures, layers, tools, and a simple ETL + model demo.

## 02 Data Preparation - practical foundations

Learn practical techniques for building ML data pipelines, cleaning data, validating datasets, and preparing a dataset in PySpark

## 03 Feature Engineering – the art of extracting value from data

Discover how to create effective features, handle different types of data, apply encoding and normalization, monitor feature drift, and track features with Spark + MLflow.

## 04 ML Algorithms - the classical approach

Get familiar with core algorithms like regression, trees, and gradient boosting, understand classification, regression, clustering, their pros and cons, and try MLlib demos in Databricks.

## 05 Model Training in Practice

Learn how to split data, perform hyperparameter tuning, scale training, and train models with code and metrics using MLflow.

## 06 Deep Learning - leveling up

Understand neural network basics, choose between TensorFlow and PyTorch, leverage GPU/TPU scaling, and build a simple CNN with visualization.

## 07 ML Pipelines - automation and CI/CD

Build repeatable ML workflows with orchestration tools, CI/CD pipelines, model registry/versioning, and a Databricks pipeline demo.

## 08 MLOps - manage your ML solution

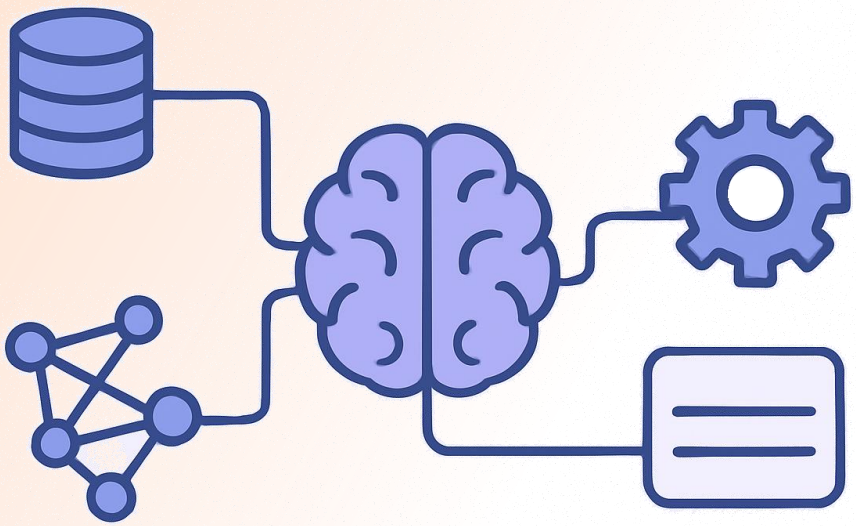
Explore ML lifecycle management with tracking, model monitoring, data drift handling, canary deployments, A/B testing, and monitoring demos.

## 09 Generative AI and LLMs - the new wave of technology

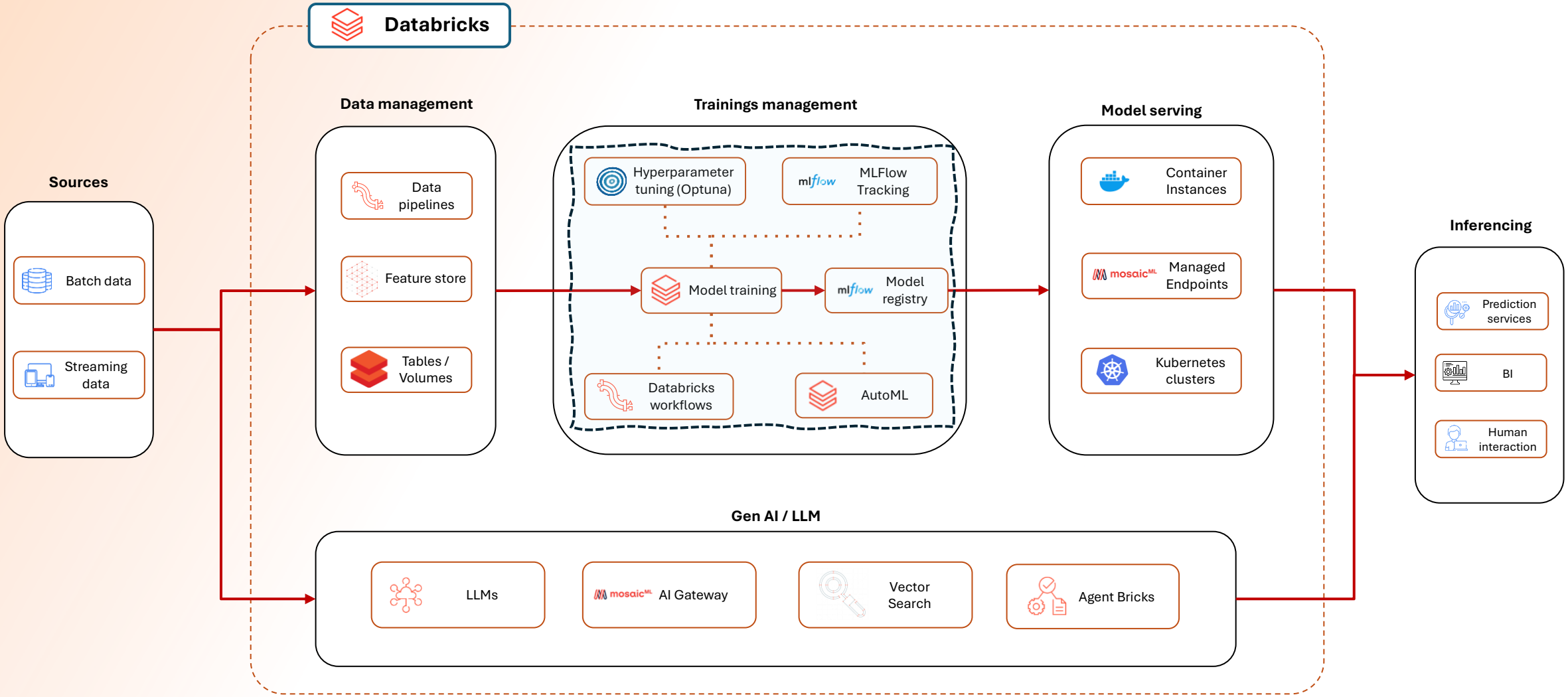
Learn how LLMs differ from classical ML, understand transformers, RAG, AI agent orchestration, and try a simple RAG demo in Azure OpenAI.

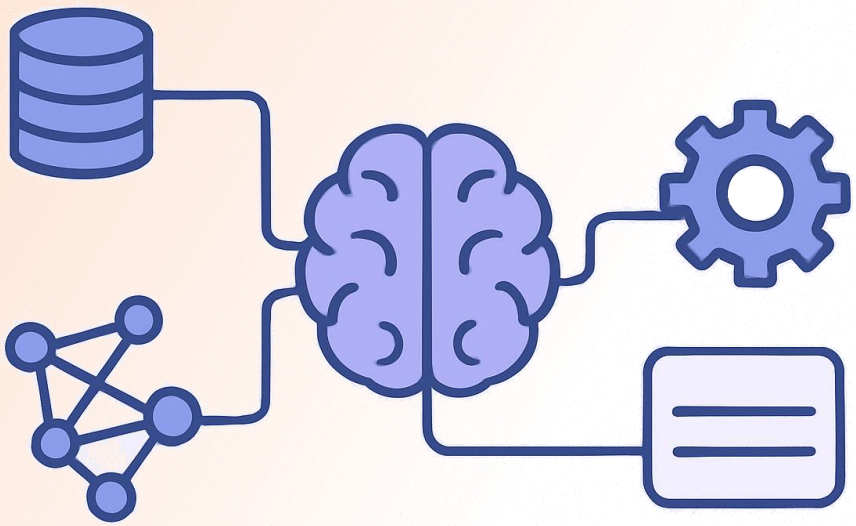
# Agenda

1. Introduction
2. Model training process
3. Hyperparameter tuning
4. Termination policies and training efficiency
5. Workshop: model training from scratch
6. Workshop: hyperparameter tuning
7. Workshop: termination policies
8. Best practices



# Introduction


Databricks
Sources
Batch data
Streaming data
Data management
Data pipelines
Feature store
Tables / Volumes
Trainings management
Hyperparameter tuning (Optuna)
MLFlow Tracking
Model training
Model registry
Databricks workflows
AutoML
Model serving
Container Instances
Managed Endpoints
Kubernetes clusters
Inferencing
Prediction services
BI
Human interaction
Gen AI / LLM
LLMs
AI Gateway
Vector Search
Agent Bricks



# Model training process

# Three-split strategy

Machine learning data must be divided into at least two (usually three) separate sets to ensure reliable model evaluation and prevent data leakage. Each set serves a distinct purpose in the training pipeline.

## Training set

Used to fit the model and learn patterns from the data. The model sees these examples repeatedly during training to adjust its internal parameters.

## Validation set

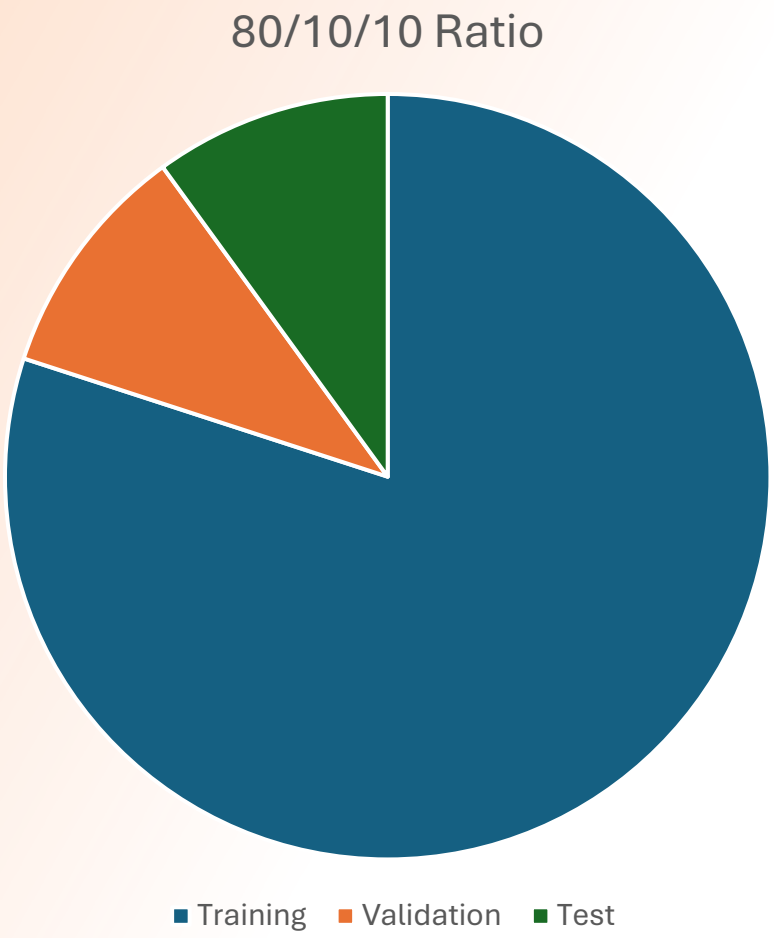
Used for intermediate evaluation and hyperparameter tuning. Helps you decide if the configuration performs well without contaminating test results.

## Test set

Used only once at the end for final, unbiased evaluation. Represents truly unseen data to measure real-world generalization performance.

⚠️ Keeping these sets separate reduces overfitting risk. The model should never "see" test data during training or tuning, it remains for the final evaluation only.

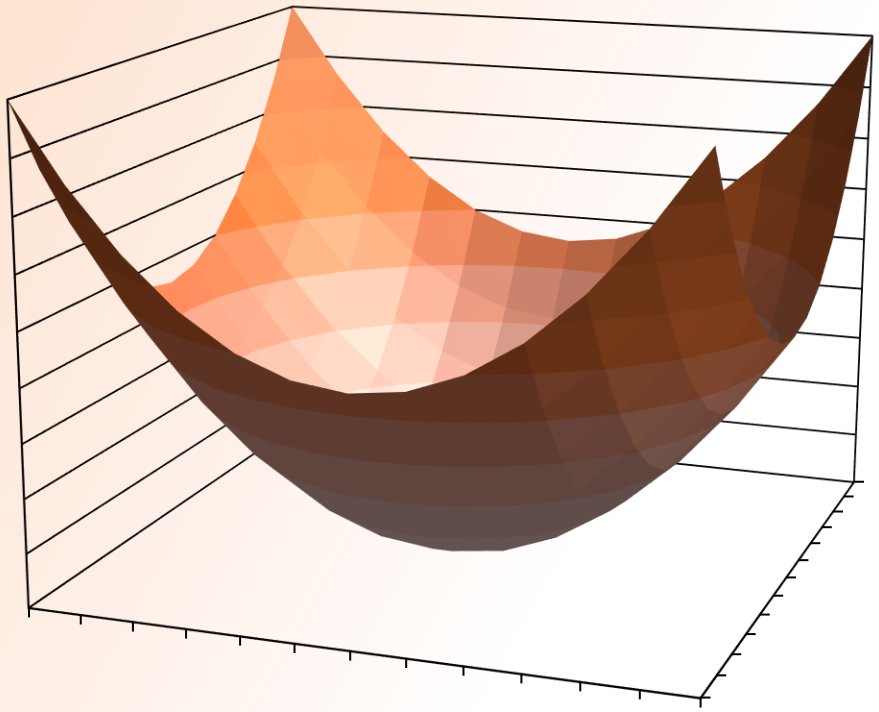
# Data split ratio




The exact proportions depend on dataset size and requirements.

For large datasets, you might allocate less to training since you have plenty of samples. For small datasets, you may need to use cross-validation instead of fixed splits to maximize data utilization.

# Loss function



A cost function quantifies the error between a model's predicted output and the true actual output. It provides a numerical measure of how well the model is performing, with a lower cost indicating better accuracy and reduced discrepancy. During training, optimization algorithms use this value to intelligently adjust the model's internal parameters, guiding it towards making more accurate predictions.

Different types of machine learning problems utilize specific cost functions, for example **Mean Squared Error (MSE)** is common for regression tasks, while **Cross-Entropy** is frequently used for classification.

# Optimization algorithms

Optimization algorithms in machine learning are used to minimize the loss function by iteratively updating the model's weights. They act as the "engine" of training, navigating the multidimensional error surface to find the point where the model's predictions are most accurate.

While basic methods like Gradient Descent move in a fixed direction based on the steepest slope, advanced optimizers like Adam use adaptive techniques (such as momentum and scaling) to speed up convergence and avoid getting stuck in local minima.

## Stochastic Gradient Descent (SGD)

- Estimates gradient using a single example or small mini-batch instead of the entire dataset.
- Use for very large datasets to speed up training; can help escape local minima due to noisy updates.

## RMSprop

- Uses an exponentially weighted moving average of squared gradients to prevent aggressive learning rate decay.
- Good for non-stationary objectives and preventing oscillations; allows larger learning rates for faster convergence.

## Adam (Adaptive Moment Estimation)

- Adapts learning rates for each parameter individually, not a fixed global rate.
- Widely effective for deep learning; often converges faster and performs well in complex landscapes.

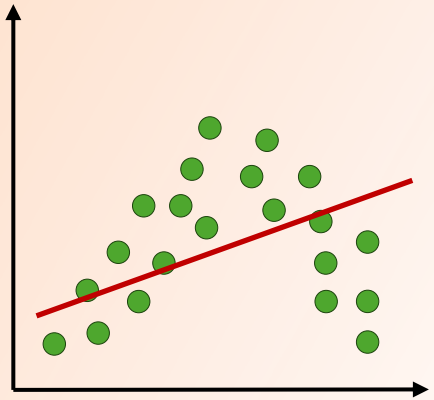
## AdaGrad

- Adapts learning rate per parameter; smaller updates for frequent features, larger for infrequent.
- Best for sparse data, giving higher learning rates to infrequent features.

⚠️ Choosing the right optimizer can significantly impact training time and the quality of the final model, often requiring experimentation.

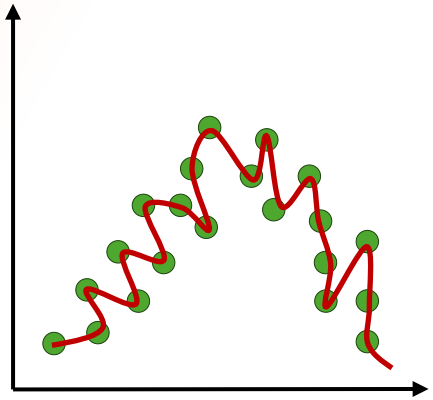
# Bias-variance trade-off

Achieving the right balance in model complexity is crucial for generalization. Here's a look at three key states of model fit, illustrated with conceptual data representations.



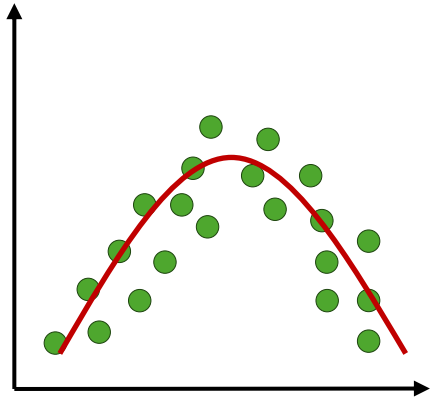
### Underfitting (high bias)

This occurs when a model is too simple to capture the underlying patterns in the training data. It results in poor performance (high error) on both the training and test sets, as the model hasn't learned enough from the data.



### Overfitting (high variance)

This happens when a model learns the training data too well, including its noise and random fluctuations. While it achieves excellent performance on the training data, its performance degrades significantly on unseen test data, failing to generalize.



### Optimal Fit

This represents the ideal scenario where the model captures the significant patterns in the training data without being overly complex. It achieves a good balance of performance on both training and test sets, indicating strong generalization capabilities.

⚠️ Aiming for optimal fit is important. An underfit model is too simple to be useful, while an overfit model is too specific to the training data to perform reliably in real-world applications.

# Model training process

1. **Initialize Model**

   Start with a model architecture and randomly initialize its parameters (weights and biases). This provides a baseline for the learning process.

2. **Forward Pass (Prediction)**

   Feed a batch of data from the training set through the model. The model makes predictions based on its current parameters.

3. **Calculate Loss (Error)**

   Compare the model's predictions with the actual target values using a predefined loss function. This function quantifies the discrepancy, indicating how far off the predictions were.

4. **Backward Pass (Update Weights)**

   Use an optimization algorithm to calculate the gradients of the loss function with respect to the model's parameters and update the weights. These gradients indicate the direction and magnitude to adjust parameters to reduce the loss.

5. **Iterate**

   Repeat steps 2-4 over many iterations (epochs) using different batches of the training data. The model continuously refines its parameters, gradually learning the underlying patterns and improving its predictive accuracy.

# Performance metrics

Different machine learning tasks require specific evaluation metrics to effectively measure model performance.

## Classification Metrics

- **Accuracy:** Proportion of correct predictions.
  - **Use when:** Classes are balanced and all prediction errors have similar impact.
  - **Avoid when:** Classes are imbalanced, as it can be misleading.

- **Precision:** Proportion of positive predictions that are actually correct.
  - **Use when:** Minimizing false positives is critical (e.g., spam detection).
  - **Avoid when:** False negatives are equally or more costly.

- **Recall:** Proportion of actual positives correctly identified.
  - **Use when:** Minimizing false negatives is critical (e.g., disease detection).
  - **Avoid when:** False positives are highly undesirable.

- **F1-score:** Harmonic mean of precision and recall.
  - **Use when:** Both precision and recall are important, especially with imbalanced classes.
  - **Avoid when:** One metric (precision or recall) is clearly more critical than the other.

- **ROC-AUC:** Measures trade-off between true positive and false positive rates.
  - **Use when:** Evaluating classifier performance across all possible thresholds, particularly with imbalanced data.
  - **Avoid when:** You need a simple, interpretable single threshold performance measure.

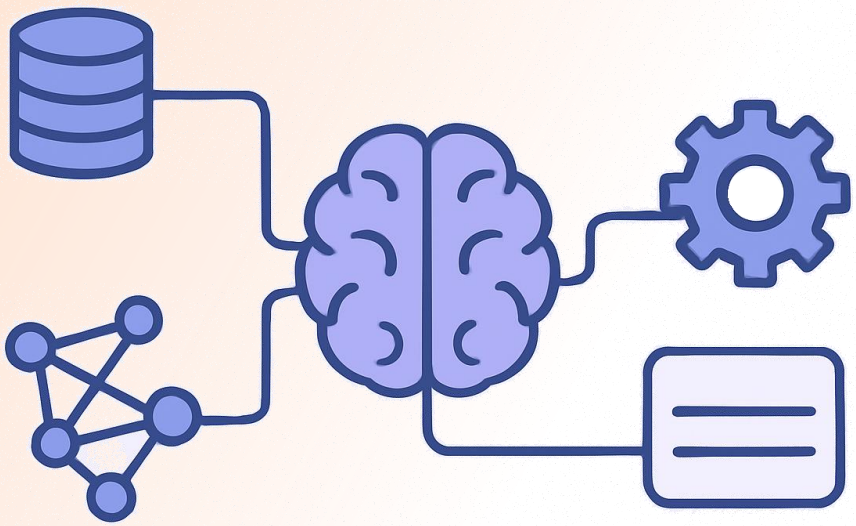
## Regression Metrics

- **RMSE:** Root Mean Squared Error, penalizes large errors more heavily.
  - **Use when:** Large errors are particularly undesirable due to their significant impact.
  - **Avoid when:** Outliers are present and their impact should not disproportionately influence the metric.

- **MAE:** Mean Absolute Error, treats all errors equally.
  - **Use when:** Errors should be treated linearly, and outliers should not have excessive influence.
  - **Avoid when:** Larger errors are significantly more critical than smaller ones.

- **$R^2$:** Coefficient of determination, proportion of variance explained by the model.
  - **Use when:** You want to understand how well the model predicts the variability of the dependent variable.
  - **Avoid when:** Evaluating absolute error, as it measures goodness of fit, not prediction accuracy.

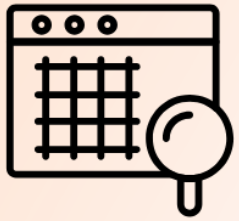
- **MSE:** Mean Squared Error, often used as an optimization objective.
  - **Use when:** You need a differentiable loss function for gradient-based optimization; large errors are costly.
  - **Avoid when:** Interpreting the error in the same units as the target variable (use RMSE instead).



# Hyperparameter tuning

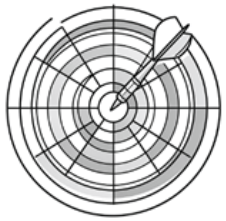
# Hyperparameter Tuning Approaches

Hyperparameter tuning aims to find the best configuration of model settings that control the learning process itself - learning rate, tree depth, number of layers, regularization strength, and so forth.



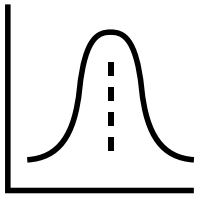
## Grid Search

Evaluates all possible combinations from a predefined parameter grid. Exhaustive but computationally expensive. Use when parameter space is small and you want to guarantee finding the optimal combination.



## Random Search

Samples random combinations from parameter space. Often more efficient when space is large since not all parameters matter equally. Available in scikit-learn as RandomizedSearchCV.



## Bayesian Optimization

Uses probabilistic models to predict promising hyperparameter combinations. More intelligent exploration of parameter space. Libraries: Optuna, Ray Tune, Hyperopt.

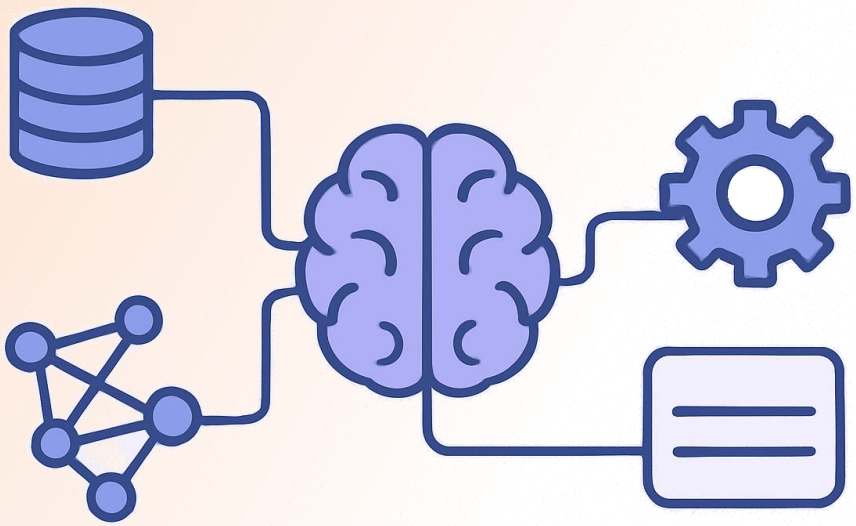
# Advanced Tuning: Optuna Example

```python
import optuna
import xgboost as xgb
from sklearn.model_selection import cross_val_score

def objective(trial):
    # Define hyperparameter search space
    n_estimators = trial.suggest_int('n_estimators', 50, 300)
    max_depth = trial.suggest_int('max_depth', 2, 8)
    learning_rate = trial.suggest_float('learning_rate', 0.01, 0.3)

    # Create and evaluate model
    model = xgb.XGBClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        learning_rate=learning_rate,
        random_state=42,
        enable_categorical=True,
        eval_metric='logloss'
    )

    # Cross-validation score
    scores = cross_val_score(model, X_train, y_train, cv=5)
    return scores.mean()

# Run optimization
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=5)

# Get best parameters
best_params = study.best_params
```

Optuna is a powerful framework for hyperparameter optimization using Bayesian methods.

The objective function defines the machine learning task and how its performance is evaluated, using the trial object to define the hyperparameter search space.

Inside the objective, a model is instantiated with suggested hyperparameters and evaluated using selected method for robust assessment.



# Termination policies and training efficiency

# Smart training termination

Early stopping monitors validation performance during training and stops automatically if improvements plateau. This prevents overfitting, saves computation time, and eliminates the need to manually choose the number of epochs.

01

## Monitor Validation

Track performance metric (accuracy, loss, etc.) on validation set after each epoch

02

## Track Best Model

Save model checkpoint whenever validation metric improves

03

## Count Patience

Track consecutive epochs without improvement up to patience threshold

04

## Stop & Restore

Stop training when metric doesn't improve for patience epochs and restore best model

### Parameters to Set

- **Patience** - How many epochs to wait without improvement
- **Min Delta** - Minimum change to qualify as improvement
- **Restore Best** - Whether to restore the best model weights

# Training efficiency

The choice between single-node and distributed training depends on your dataset size, model complexity, and available resources. Each approach has distinct advantages and considerations.

## Single-Node Training

**Use when:** Small to medium datasets and models

**Hardware:** CPU or single GPU

**Advantages:** Simpler setup • No network overhead • Lower infrastructure cost • Easier debugging

**Mechanics:** Data batches flow sequentially through a single GPU/CPU. The model's parameters and intermediate activations must fit within the device's memory. Training speed is constrained by the device's computational throughput and memory bandwidth, leading to limits on model size and batch size.

## Distributed Training

**Use when:** Very large datasets and deep neural networks

**Hardware:** Multi-node GPU cluster

**Advantages:** Drastically reduced training time • Scales beyond single machine limits • Handles massive datasets

**Mechanics:** Involves multiple processing units (nodes/GPUs) working in parallel. Key strategies include **data parallelism** (each node gets a subset of the data batch), **model parallelism** (different parts of the model are placed on different nodes) and **synchronization** (ensuring that all nodes are aligned before proceeding)

# Right training strategy

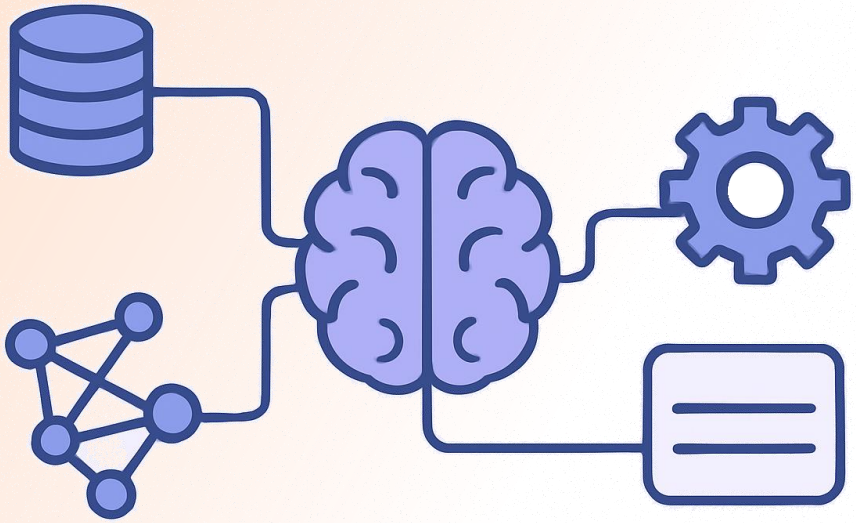
Use **single-node training** for simplicity and cost-efficiency when your workload is moderate. The overhead of distributed training often outweighs benefits for smaller problems.

Use **distributed training** when scale makes it necessary - training deep networks on millions of images, processing massive datasets, or when training time needs to be hours rather than days or weeks.

**Remember:** Distributed training introduces communication overhead, synchronization costs, and network bandwidth limitations. For small datasets or models, this overhead can actually slow you down.

⚠️ The key is matching your training infrastructure to your problem scale. Start simple, measure performance, and scale up only when necessary. Smart choices here save time, money, and frustration.

# DEMO



# Summary

# Takeaways

**Implement a Robust Data Strategy:** Divide your dataset into training, validation, and test sets to ensure reliable evaluation and prevent data leakage. Use the validation set for hyperparameter tuning and keep the test set strictly for a final, unbiased assessment of real-world generalization.

**Master the Bias-Variance Trade-off:** Actively diagnose and mitigate underfitting by increasing model capacity or improving feature engineering, while preventing overfitting through automated early stopping, regularization, and the use of independent validation sets to ensure high generalization performance on production data.

**Prioritize Scalability and Resource Efficiency:** Design training infrastructure that matches the problem scale. Start with centralized processing for efficiency and pivot to distributed training only when handling massive datasets or complex deep learning architectures to balance cost and performance.

# Thank you!

**Contact:**

https://www.linkedin.com/in/maciej-kepa

https://github.com/maciejkepa/ai-ml-in-practice